3.1 ServiceNow Testing

Document 3.1 — ServiceNow Testing

Project: Hospital Appointment Management System **Platform:** ServiceNow (Scoped Application) **Application Scope:** x_2065601_hospit_0 **Phase:** Project Development — Testing

---

1. Test Strategy Overview

Testing for the Hospital Appointment Management System was carried out across three layers, reflecting the platform-specific risks identified during planning:

- **Functional Testing:** Verifying that each core workflow (registration, slot creation, booking, cancellation, waitlist promotion, no-show handling) behaves correctly end-to-end and that portal actions are correctly reflected in the underlying ServiceNow tables.
- **ACL / Role Testing:** Verifying that Patient, Doctor, and Front Desk/Admin roles can only access the data and actions appropriate to their persona, and that no role can view or modify records outside its intended scope.
- **Concurrency Testing:** Verifying that the atomic slot booking transaction correctly prevents double-booking when two booking attempts target the same slot in close succession.

All test cases were executed against the scoped application in a sub-production ServiceNow instance, with results cross-verified directly in the platform's list views for the Patient, Doctor, Doctor Availability Slot, Appointment, and Waitlist tables.

---

2. Test Case Table

Test ID	Test Scenario	Steps	Expected Result	Actual Result	Status
TC-01	Patient registration	Navigate to Patient Registration tab; fill in name, age, blood group, phone, city, address, primary issue; submit	New record created in Patient table with all fields correctly populated	Record created and visible in Patient table with matching field values	Pass
TC-02	Doctor registration	Navigate to Doctor Registration tab; fill in specialization, qualification, max patients per day, shift, consultation fee, slot time; link to sys_user; submit	New record created in Doctor table, correctly linked to the sys_user reference	Record created and visible in Doctor table with correct sys_user reference	Pass
TC-03	Doctor adds availability slot	As a doctor, navigate to Doctor Queue & Slots tab; add a new slot with date, start time, end time	New record created in Doctor Availability Slot table with status "Available"	Slot record created with status "Available" as expected	Pass
TC-04	Patient books an available slot	As a patient, browse available slots for a doctor; select a slot; confirm booking	Appointment record created with state "Booked"; slot status flips to "Booked" in the same transaction	Appointment created and slot status updated correctly in a single transaction	Pass
TC-05	Double-booking prevention	Two simulated booking attempts submitted in quick succession for the same slot	Only the first request succeeds; the second is rejected with a "slot no longer available" message	First request succeeded; second request correctly rejected, no duplicate appointment created	Pass
TC-06	Appointment	As a patient, open	Appointment state	Appointment	Pass

TC-06	Appointment cancellation	As a patient, open Booking & History tab; select an upcoming appointment; cancel it	Appointment state changes to “Cancelled”; associated slot status reverts to “Available”	Appointment marked “Cancelled”; slot status correctly reverted to “Available”	Pass
TC-07	Waitlist auto-promotion	Patient A on waitlist for a doctor/date; Patient B cancels their booked appointment for that date	Waitlist record for Patient A is promoted; Patient A’s status becomes “Promoted” and an appointment is created	Waitlist status updated to “Promoted” and appointment auto-created for Patient A	Pass
TC-08	No-show flagging	As a doctor, mark a checked-in-but-absent patient as “No-Show” in the daily queue	Appointment state updates to “No-Show”; slot is freed for waitlist promotion logic	Appointment state updated correctly; slot freed and waitlist promotion triggered as in TC-07	Pass
TC-09	ACL / role restriction	As a Patient-role user, attempt to access the Front Desk/Admin tab or another patient’s appointment record directly	Access denied; patient can only view their own records	Access correctly denied by ACL; only own records visible	Pass
TC-10	Front-desk manual booking	As Front Desk/Admin, use the Front Desk/Admin tab to book a walk-in patient against an available slot	Appointment and, if needed, new Patient record created correctly on behalf of the walk-in patient	Appointment and patient record created correctly through the admin flow	Pass
TC-11	Empty-state handling	As a patient, view slots for a doctor with no availability entered for the selected date	Portal displays a clear “No slots available” message instead of an error or blank screen	Empty-state message displayed correctly, no console/UI errors	Pass

---

3. Bugs Found and Resolved

1. **Field name mismatch causing silent save failure.** During initial booking flow testing, appointment records were not being created despite the UI showing a success message. Root cause was a mismatch between the widget’s client-side field name (`slot_id`) and the actual backend field name (`slot`) on the Appointment table. **Resolution:** Aligned all client script and server script field references to the exact backend field names, and added server-side validation to surface an error if a required field is missing rather than failing silently.
2. **Waitlist promotion not triggering on no-show.** Automated promotion correctly triggered on manual cancellation but did not trigger when a patient was marked as “No-Show,” because the triggering logic was initially scoped only to the Cancelled state. **Resolution:** Updated the triggering condition to fire on any transition that results in slot status becoming “Available” (covering both Cancelled and No-Show paths), rather than hardcoding a single state.
3. **Double-booking under rapid concurrent requests.** In early testing, two near-simultaneous booking requests for the same slot occasionally both succeeded, creating two appointment records against one slot. **Resolution:** Restructured the booking logic so the slot status check and update, along with appointment creation, occur within a single atomic GlideRecord transaction rather than as separate read-then-write steps, closing the race condition window.

End of Document 3.1